Requirements

Ingegneria del Software

2025/2026

Università di Trento

Chiara Di Francescomarino

Today

- What is a software requirement?
- Types of software requirements
- Requirements engineering
- Problems with requirements
- Forms of requirements

What is a software requirement?

What is a requirement?

A desire? A need?

Objectives and functionalities

- The needs of a customer can be expressed as objectives (business objectives)
  - Example: I want to read a book
- When designing a system to achieve a goal we need to specify a set of functionalities
- The relation between objectives and functionalities that allows to achieve the objective is what is studied in the analysis of the requirements

What is a software requirement?

- What the system will do or a system constraint
- It may range from a high-level abstract statement of a service or of a system constraint to a detailed mathematical functional specification.
- Several definitions …

WHAT the system will do and
NOT HOW it will do it!

A software requirement is … (IEEE 610)

1. A condition or capability needed by a user to solve a problem or achieve an objective
2. A condition or capability that must be met or possessed by a system or component to satisfy a contract, standard, specification, or other formally imposed documents
3. A documented representation of a condition or capability as in (1) or (2)

The dual nature of a software requirement

- Requirements may serve a dual function
  - May be the basis for a bid for a contract - therefore must be open to interpretation;
  - May be the basis for the contract itself - therefore must be defined in detail;

Difference between feature and requirement

- A requirement describes the capability that the system has to possess; it is written to test the implementation in the context of the scenarios of the system
- A feature is a set of requirements logically connected; it usually describes a functionality of a product

- Feature: cart of an e-commerce website
- Requirements:
  - The user can add items to the cart
  - The user can remove items from the cart
  - ….

Examples of requirements

- For an ATM system
  - The ATM system has to check the card validity
  - The ATM system cannot return more the 250 euros in the same day

- For computing Fibonacci(n)
  - F(0) = 1
  - F(1) = 1
  - F(n) = F(n-1)+F(n-2)

Requirements: the most complex part

The most complex part of building a software system is deciding what to build

No part of the work affects the outcome more if it is done incorrectly

No other part is more difficult to correct later

Fred Brooks

Turing Award 1999

Brooks, F. 1975 (1995). *Mythical Man-month: essays on software engineering, 20th anniversary edition. Addison-Wesley Professional.*

Why are requirements important?

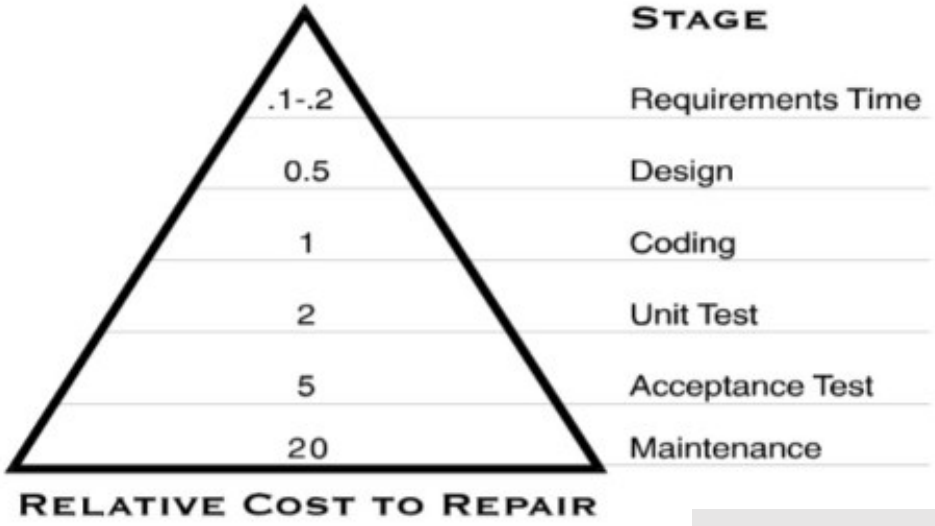
- Causes of failure of a software project (Standish survey 1994)

Project Impaired Factors	% of Responses
1. Incomplete Requirements	13.1%
2. Lack of User Involvement	12.4%
3. Lack of Resources	10.6%
4. Unrealistic Expectations	9.9%
5. Lack of Executive Support	9.3%
6. Changing Requirements & Specifications	8.7%
7. Lack of Planning	8.1%
8. Didn't Need It Any Longer	7.5%
9. Lack of IT Management	6.2%
10. Technology Illiteracy	
Other	

44.1% of the causes of failure are somehow related to the requirements!

Impact of requirements errors

As much as 20:1 cost savings results from finding errors in the requirements stage versus finding errors in the maintenance stage of the software lifecycle



Barry Boehm-
'76, 88

More than half of the bugs can be traced back to errors made during the requirements stage

Cost of fixing defects

- Errors should be discovered as soon as possible … at requirements time

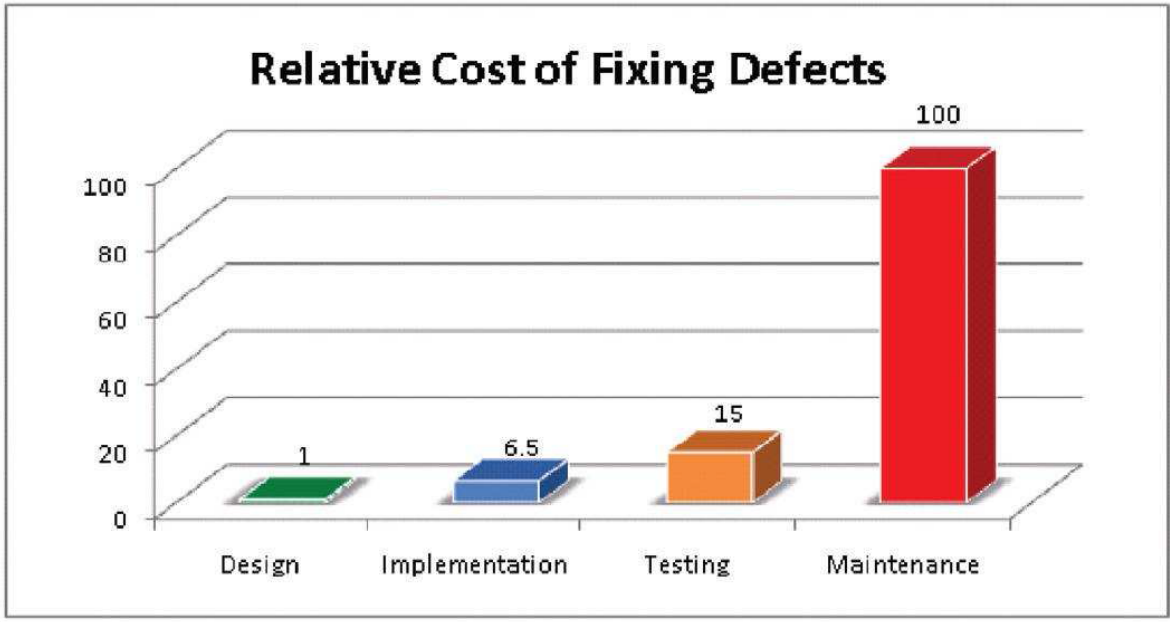


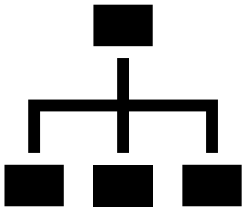
Figure 3: *IBM System Science Institute Relative Cost of Fixing Defects*

If fixing a defect at requirement time has a cost of 1, removing the same defect at testing time has a cost of 15, …

What is not a requirement?

- System architecture
- Technological constraints
- Development process
- Development environment
- Operating system
- Reusability and portability aspects

- Domain descriptions of the real world are part of the requirements



Types of requirements

User and System requirements

Different types of requirements

User requirements

- Statements in natural language plus diagrams of the services the system provides and its operational constraints. Written for customers.
- Written by and for the customer

System requirements

- A structured document setting out detailed descriptions of the system's functions, services and operational constraints. Defines what should be implemented so may be part of a contract between client and contractor.
- Written for the developers
- Different notations can be used for their description

Different types of requirements

User requirements

- Statements in natural language plus diagrams of the services the system provides and its operational constraints. Written for customers.
- Written by and for the customer

System requirements

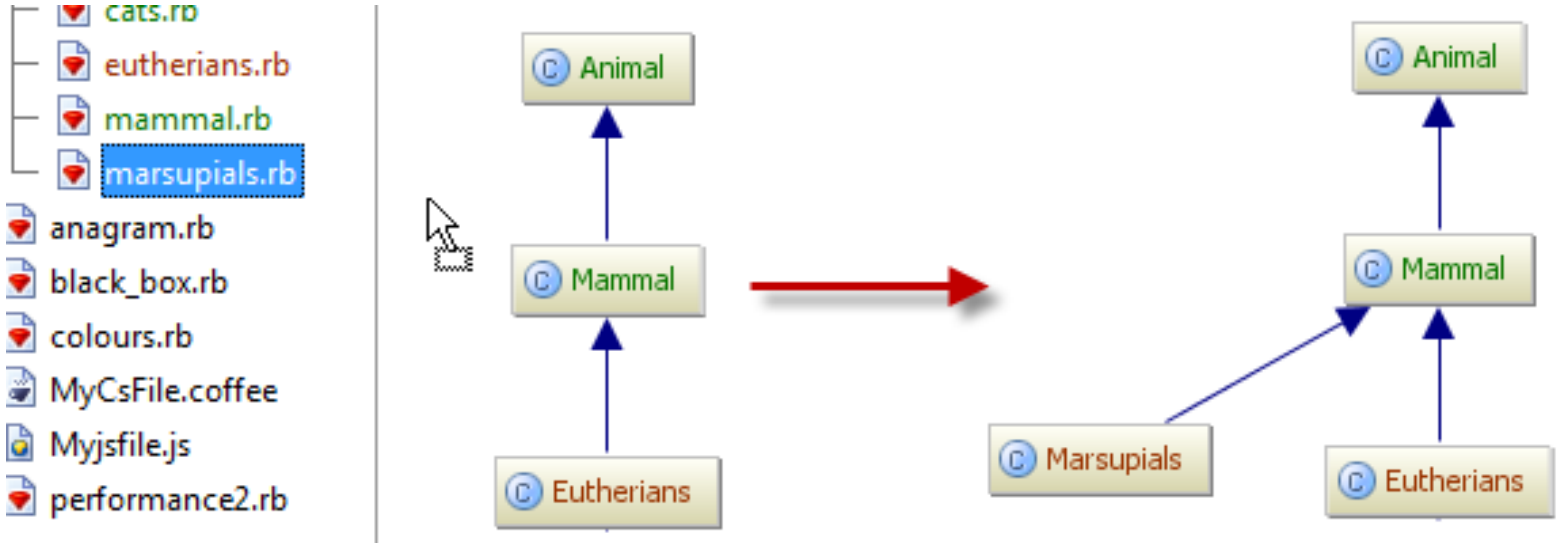
- A structured document setting out detailed descriptions of the system's functions, services and operational constraints. Defines what should be implemented so may be part of a contract between client and contractor.
- Written for the developers

can be
iption

Warning: if both of them exist they have to be aligned

Example

- We have a graphical editor and we want to describe the requirement for a functionality that allows us to add a new node



User requirement example

3.5.1 Adding nodes to a design

3.5.1.1 **The editor shall provide a facility for users to add nodes of a specified type to their design.**

3.5.1.2 The sequence of actions to add a node should be as follows:

1. The user should select the type of node to be added.
2. The user should move the cursor to the approximate node position in the diagram and indicate that the node symbol should be added at that point.
3. The user should then drag the node symbol to its final position.

Rationale: The user is the best person to decide where to position a node on the diagram. This approach gives the user direct control over node type selection and positioning.

Specification: ECLIPSE/WS/Tools/DE/F

Written for the user

System requirement example

ECLIPSE/Workstation/Tools/DE/FS/3.5.1	
Function	Add node
Description	Adds a node to an existing design. The user selects the type of node, and its position. When added to the design, the node becomes the current selection. The user chooses the node position by moving the cursor to the area where the node is added.
Inputs	Node type, Node position, Design identifier.
Source	Node type and Node position are input by the user, Design identifier from the database.
Outputs	Design identifier.
Destination	The design database. The design is committed to the database on completion of the operation.
Requires	Design graph rooted at input design identifier.
Pre-condition	The design is open and displayed on the user's screen.
Post-condition	The design is unchanged apart from the addition of a node of the specified type at the given position.
Side-effects	None
Definition:	ECLIPSE/Workstation/Tools/DE/RD/3

More detailed and precise
… for the developer

Another example: the Mentcare system

- For clinics attended by patients suffering from mental health problems
- Records details of their consultations and conditions.
- It has to generate letters and reports of different types and help ensure that the laws pertaining to mental health are maintained by staff treating patients.
- Among the different reports it has to produce a monthly report with the costs of the drugs prescribed by each clinic

User and system requirements example

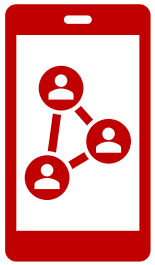
User requirement

- It shall generate monthly management reports showing the cost of drugs prescribed by each clinic during that month

System requirement

- On the last working day of each month, a summary of the drugs prescribed, their costs and the prescribing clinics shall be generated
- The printing has to be generated after 17:30 of the last working day
- …

Deeper level of detail



Functional and non-functional requirements

Functional and non-functional requirements

Functional requirements

- Functionalities and services the system should provide, how the system should react to particular inputs and how the system should behave in particular situations.
- May state what the system should not do.
- Independent of the implementation

Non-functional requirements

- Constraints on the services or functions offered by the system such as timing constraints, constraints on the development process, standards, etc.
- Often apply to the system as a whole rather than to individual features or services
- Usually about languages, tools, performance, …

Functional requirements

- Describe functionality or system services.
- Functional user requirements may be high-level statements of what the system should do.
- Functional system requirements should describe the system services in detail.
- Imprecisions in the requirement specification is the cause of many software engineering problems

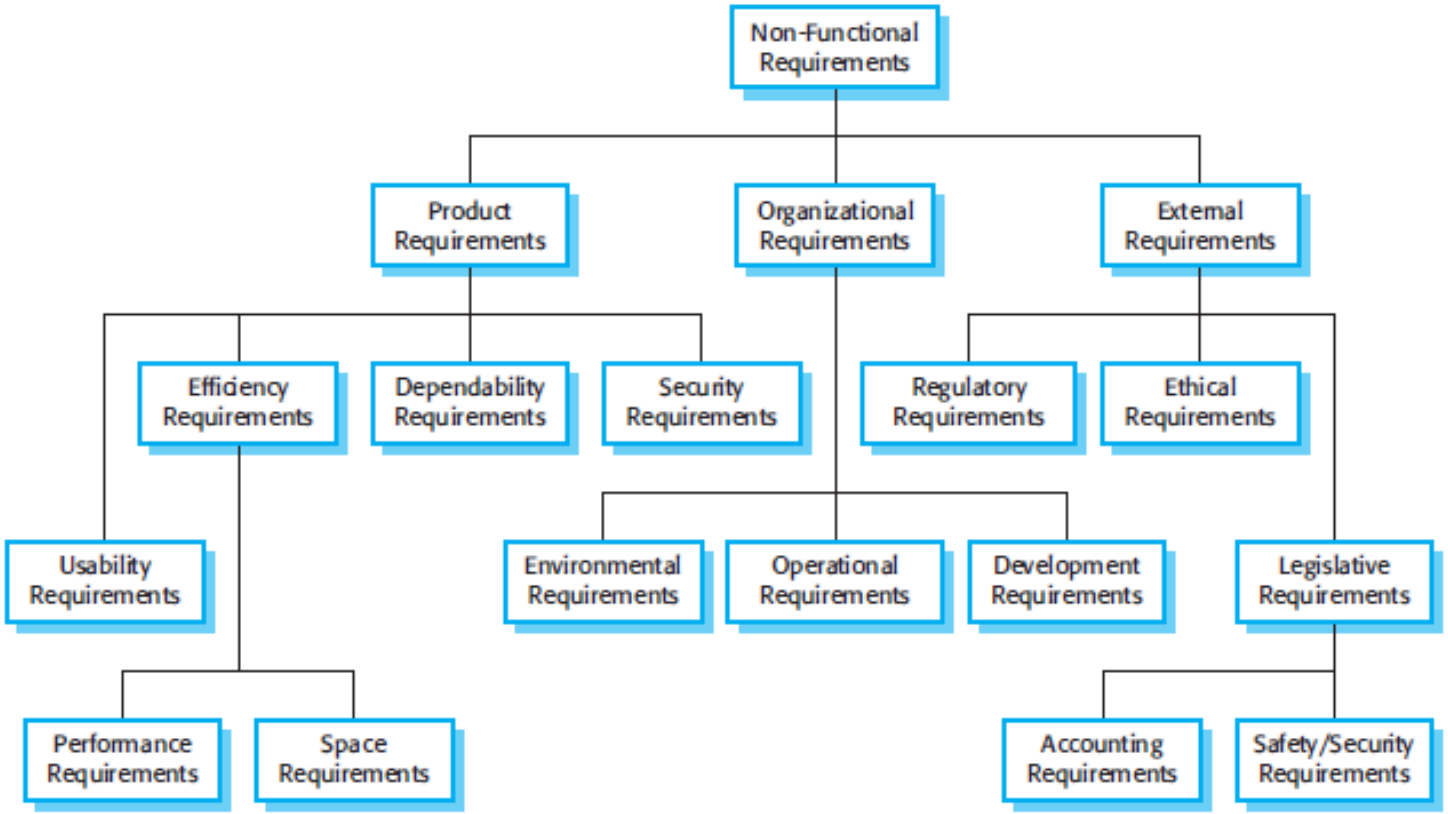
Non-functional requirements

- Define system properties and constraints e.g. reliability, response time and storage requirements. Constraints are I/O device capability, system representations, etc.
- Process requirements may also be specified mandating a particular IDE, programming language or development method.
- Non-functional requirements may be more critical than functional requirements. If these are not met, the system may be useless.

Non-functional requirements implementation

- May affect the overall architecture of a system rather than the individual components.
  - For example, to ensure that performance requirements are met, you may have to organize the system to minimize communications between components.
- A single non-functional requirement, such as a security requirement, may generate a number of related functional requirements that define system services that are required.
  - It may also generate requirements that restrict existing requirements.

Non-functional requirements taxonomy



Non-functional requirements classification

- Product requirements -> product
  - Requirements which specify that the delivered product must behave in a particular way e.g. execution speed, reliability, etc.
- Organisational requirements -> development process
  - Requirements which are a consequence of organisational policies and procedures e.g. process standards used, implementation requirements, etc.
- External requirements
  - Requirements which arise from factors which are external to the system and its development process e.g. interoperability requirements, legislative requirements, etc.

Exercise: banking system

- Which ones among the following requirements related to a banking system are functional requirements and which ones are non-functional ones?

- Documents should be registered in the PDF format
- The system shall allow to change the balance from euros to dollars when asked by the customer
- The answer to a query has to be returned within 3 seconds
- The system shall allow for checking the balance
- PCs have to be IBM PCs

Exercise: Mentcare system

- Which ones are functional and which ones are non-functional requirements?

- A user shall be able to search the appointments lists for all clinics
- The system shall be available to all clinics during normal working hours (Mon–Fri, 08.30–17.30). Downtime within normal working hours shall not exceed five seconds in any one day.
- The system shall implement patient privacy provisions as set out in HStan-03-2006-priv.
- The system shall generate each day, for each clinic, a list of patients who are expected to attend appointments that day.
- Each staff member using the system shall be uniquely identified by his or her 8-digit employee number.
- Users of the system shall authenticate themselves using their health authority identity card.

Exercise: Mentcare system

- What kind of non-functional requirements are?

- The system shall be available to all clinics during normal working hours (Mon–Fri, 08.30–17.30). Downtime within normal working hours shall not exceed five seconds in any one day.
- The system shall implement patient privacy provisions as set out in HStan-03-2006-priv.
- Users of the system shall authenticate themselves using their health authority identity card.

Exercise: Functional and non-functional requirements for ATM

Consider an ATM system.

- The system must provide the withdrawal, balance and transaction record printing functions.
- The system must be available to people with disabilities, must guarantee a response time of less than a minute, and must be developed on the X86 architecture with an operating system compatible with that of the Bank.
- Withdrawal operations must require authentication via a secret code stored on the card.
- The system must be easily expandable, and adaptable to future banking needs. …

Goals and requirements

- Non-functional requirements may be very difficult to state precisely and imprecise requirements may be difficult to verify.
- Goal
  - A general intention of the user such as ease of use.
- Verifiable non-functional requirement
  - A statement using some measure that can be objectively tested.
- Goals are helpful to developers as they convey the intentions of the system users.

Quantifying non-functional requirements

- The system should be easy to use by medical staff and should be organized in such a way that user errors are minimized. (Goal)
- Medical staff shall be able to use all the system functions after four hours of training. After this training, the average number of errors made by experienced users shall not exceed two per hour of system use. (Testable non-functional requirement)

Metrics for specifying non-functional requirements

Property	Measure
Speed	Processed transactions/second User/event response time Screen refresh time
Size	Mbytes Number of ROM chips
Ease of use	Training time Number of help frames
Reliability	Mean time to failure Probability of unavailability Rate of failure occurrence Availability
Robustness	Time to restart after failure Percentage of events causing failure Probability of data corruption on failure
Portability	Percentage of target dependent statements Number of target systems

Let’s try a test

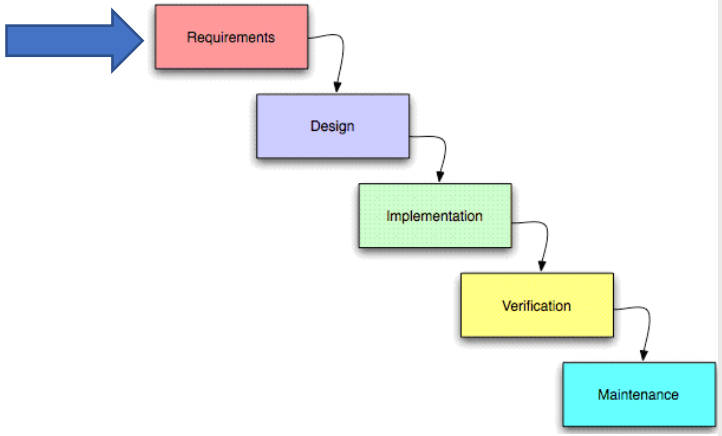
How to participate?

Copy participation link

1 Go to wooclap.com

2 Enter the event code in the top banner

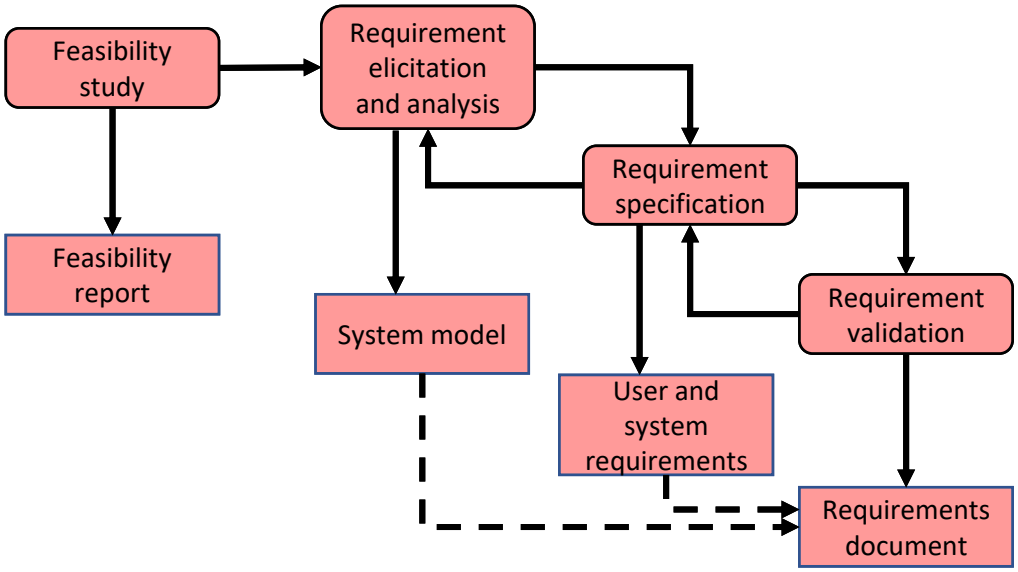
Event code
QFTIMA



Requirements engineering

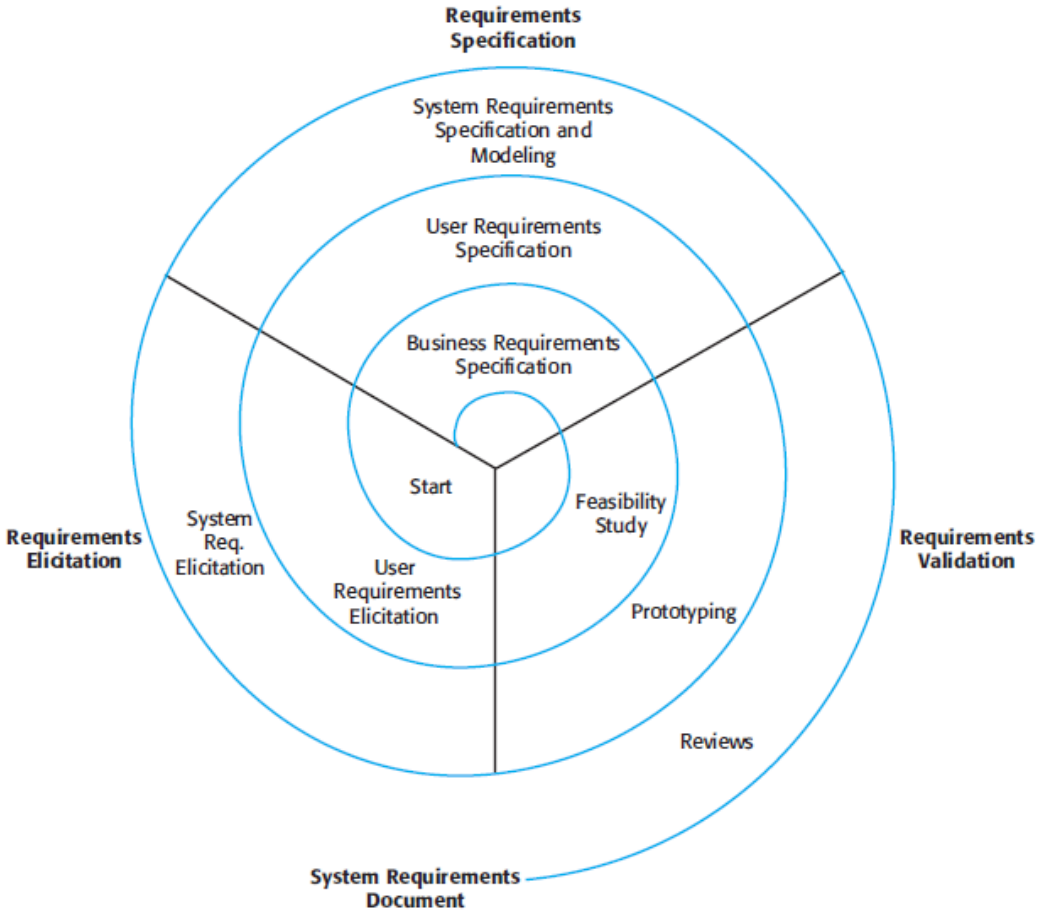
Requirements Engineering

- Describes the activities needed to collect, document and maintain the set of requirements of a software system



Requirements Engineering

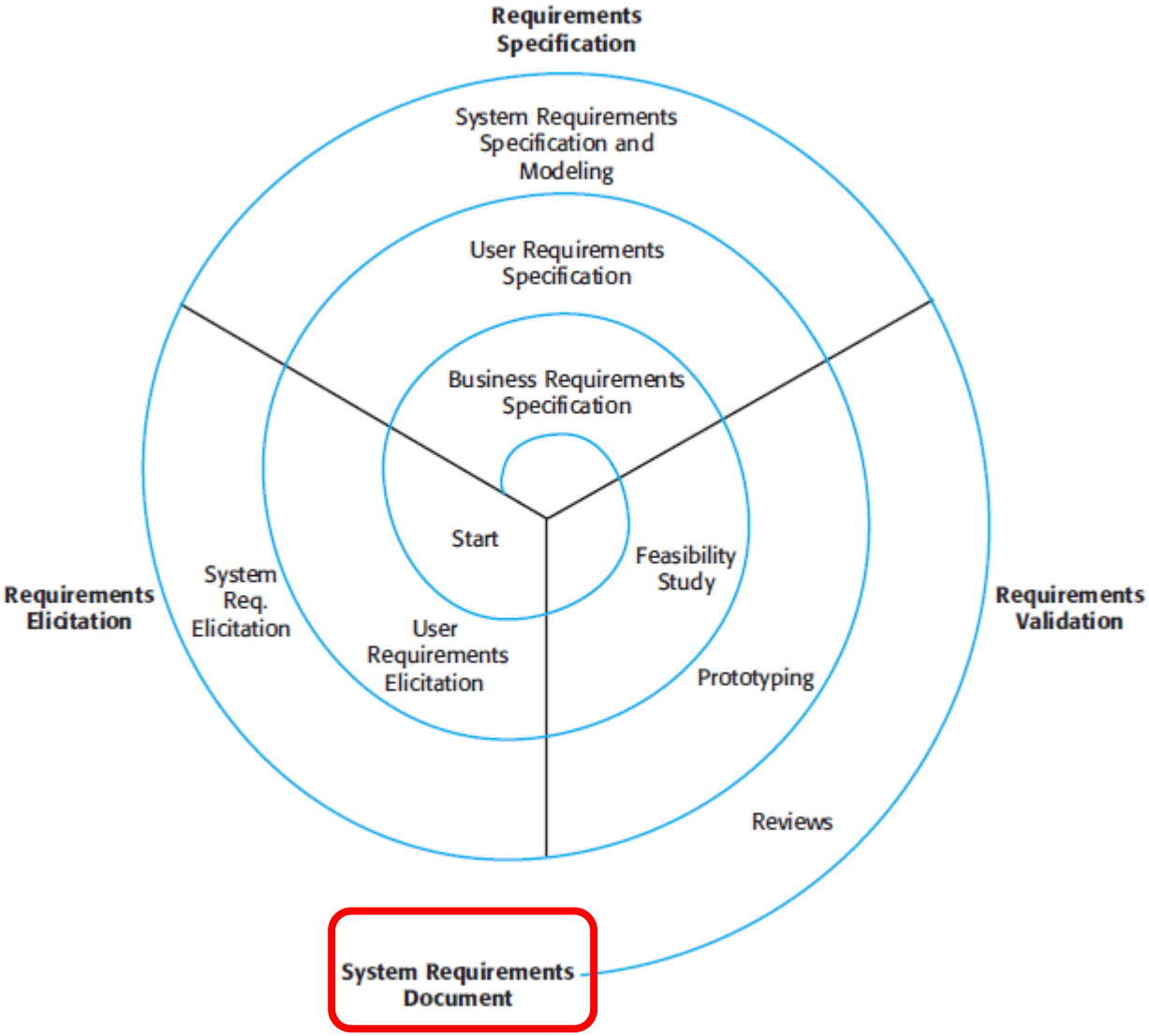
- The process of establishing the services that a customer requires to a system and the constraints
- The output is the software requirements document



What is the purpose of requirements engineering?

- The main purpose is producing the software requirement document defining functionalities and services of the system that has to be realized (and keeping it updated).

Requirements Engineering

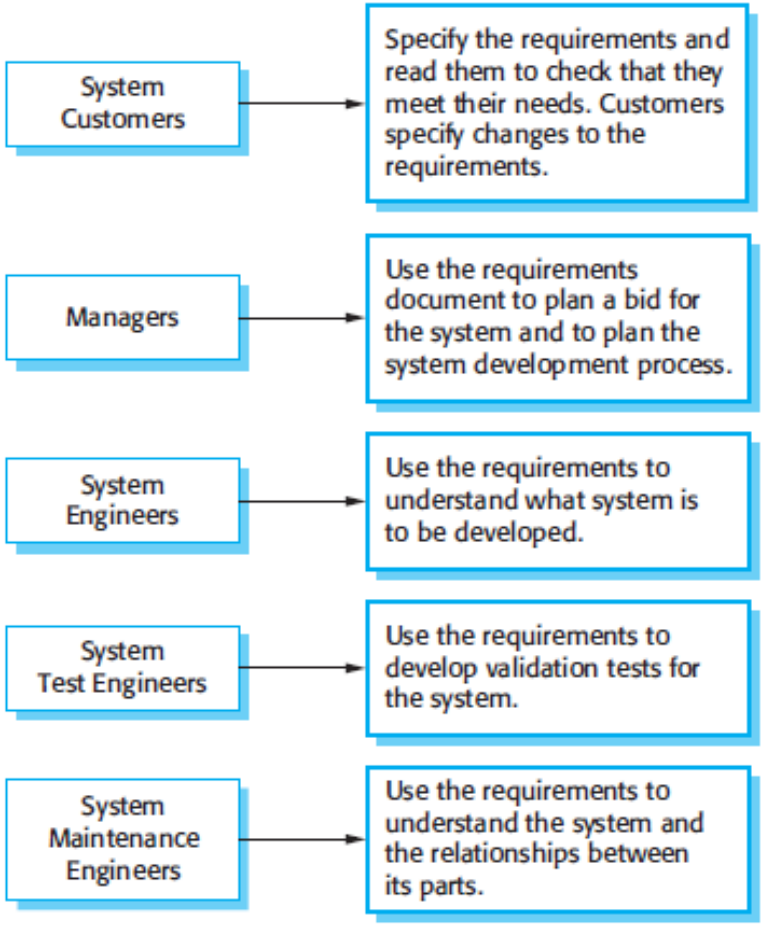


Software Requirements Document

The Software Requirements (Specification) Document (SRS)

- Official statement of what is required
- It usually includes both a definition of user requirements and a specification of the system requirements
- It is NOT a design document. It should set WHAT the system should do rather than HOW it should do it.

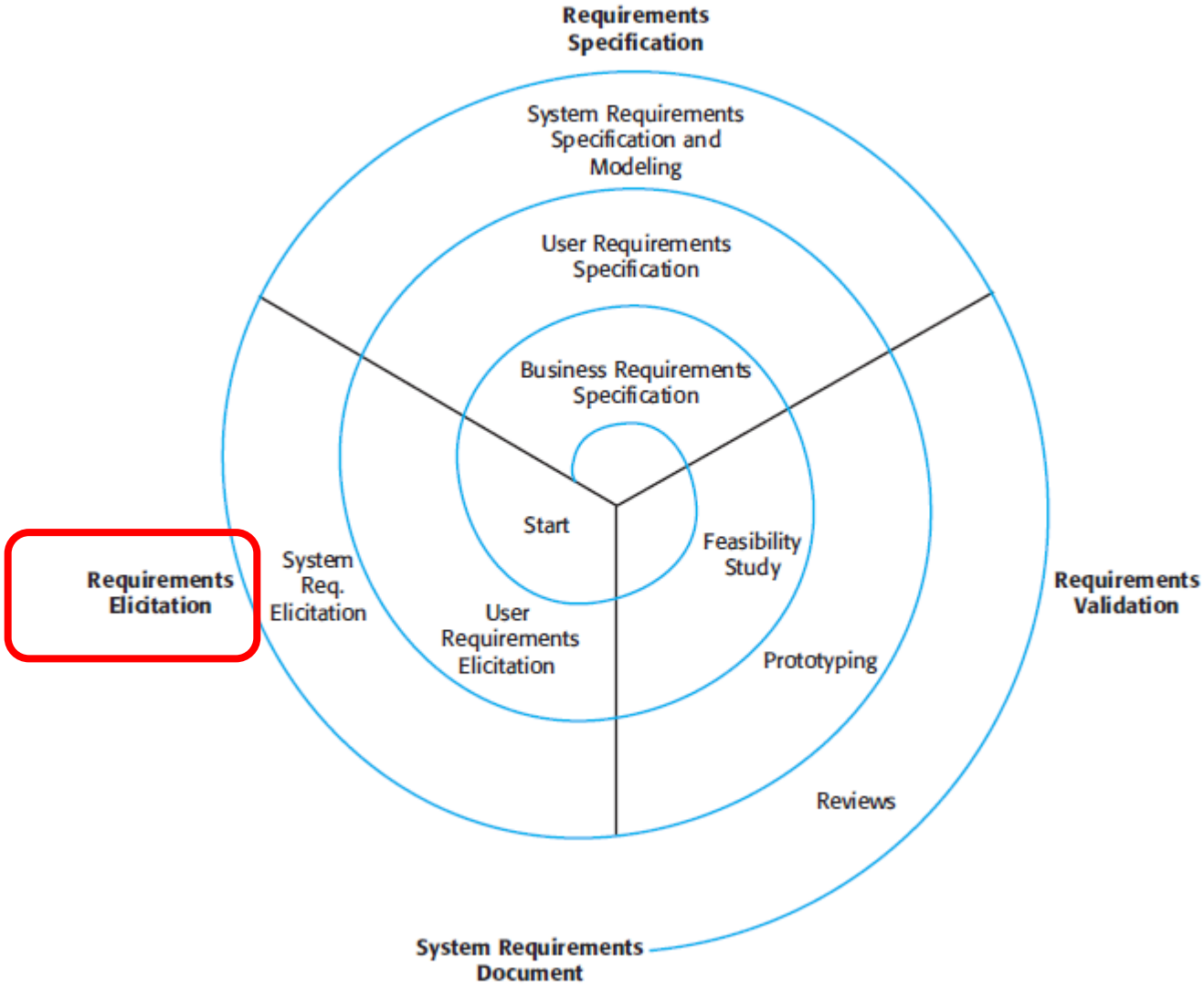
Users of a requirement document



Requirements document variability

- Information in the requirements document depends on the type of system and the approach to development used.
- Critical and outsourced systems demand for detailed requirements
- Systems developed incrementally will, typically, have less details in the requirements document.
- Requirements documents standards have been designed e.g. IEEE standard. These are mostly applicable to the requirements for large systems engineering projects.

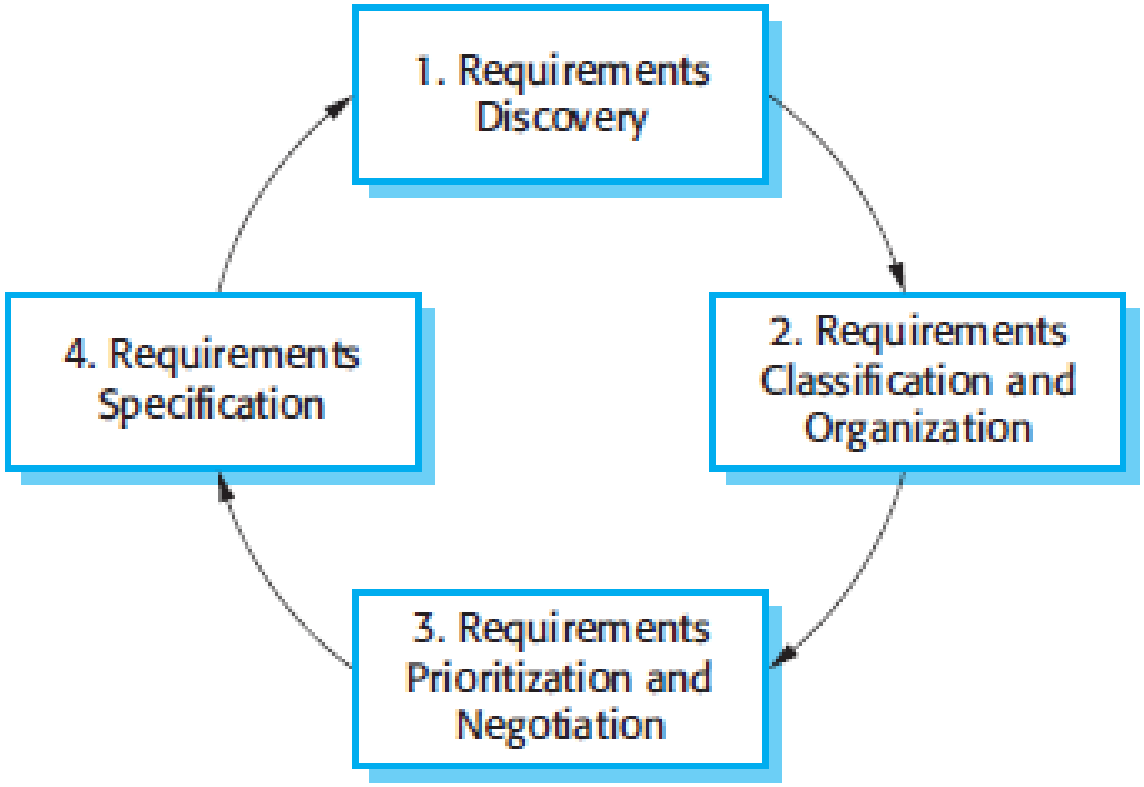
Requirements Engineering



Requirements elicitation and analysis

- Sometimes called requirement discovery
- Involves technical staff working with customers to find out about the application domain, the services that the system should provide and the system's operational constraints.
- May involve end-users, managers, engineers involved in maintenance, domain experts, trade unions, etc. These are called stakeholders.

The requirements elicitation and analysis process



Process activities

Requirements discovery	• Interacting with stakeholders to discover their requirements. Domain requirements are also discovered at this stage.
Requirements classification and organisation	• Groups related requirements and organises them into coherent clusters.
Prioritisation and negotiation	• Prioritising requirements and resolving requirements conflicts.
Requirements specification	• Requirements are documented and input into the next round of the spiral.

Requirement elicitation

Requirement elicitation

- From *elìcere:* to capture/extract the requirements from the customers
- The first step is the stakeholder identification …
- Techniques used:
  - Interviews
  - In-place observations
  - Competitors' product analysis
  - Workshops

Interviewing

- Formal or informal interviews with stakeholders are part of most RE processes.
- Types of interview
  - Closed interviews based on pre-determined list of questions
  - Open interviews where various issues are explored with stakeholders.

Interviews in practice

- Normally a mix of closed and open-ended interviewing.
- Interviews are good for getting an overall understanding of what stakeholders do and how they might interact with the system.
- Interviews are not good for understanding domain requirements
  - Requirements engineers cannot understand specific domain terminology;
  - Some domain knowledge is so familiar that people find it hard to articulate or think that it isn't worth articulating.

Effective interviews

- Be open-minded, avoid pre-conceived ideas about the requirements and are willing to listen to stakeholders.
- Prompt the interviewee to get discussions going using a springboard question, a requirements proposal, or by working together.

Problems with requirements elicitation

- Stakeholders don’t know what they really want.
- Stakeholders express requirements in their own terms.
- Different stakeholders may have conflicting requirements.
- Organisational and political factors may influence the system requirements.
- The requirements change during the analysis process. New stakeholders may emerge and the business environment may change.

Requirement analysis

- Classification
- Organization
- Prioritization

Requirement analysis

- The user needs of the stakeholders collected during the elicitation phase are analyzed and refined
- The requirements are analyzed to establish their feasibility and correctness
  - We try to understand if the requirements are correct w.r.t the customer's wishes
  - We try to identify the "missing requirements"
  - Unclear requirements are identified
  - “Contradictory or conflicting” requirements are resolved
- The priority (prioritization) is also established

Conflicting requirements example

- Hardware system “XYZ”
- Req 12. In order to minimize energy consumption, the number of chips used must be minimized, preferring low-power ones
- Req 23. Very rapid response times must be guaranteed

Problem: few low-power chips can't guarantee fast response time ...

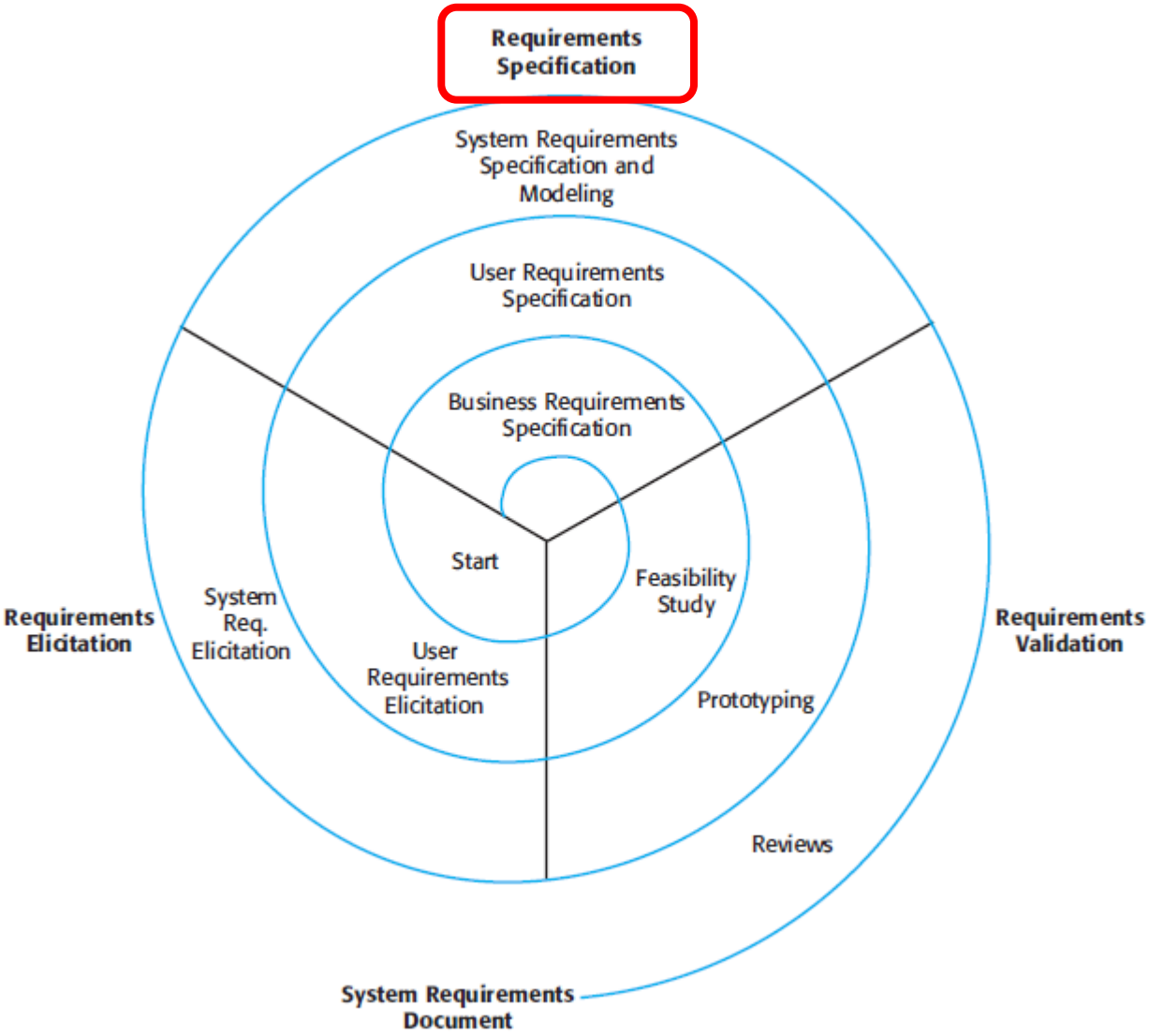
Solution: negotiation with stakeholders and removal of one of the conflicting requirements

Requirement prioritization

- To know what to "cut" if not all of them can be achieved
- To determine what to include in an increment in case of incremental development methods
- Two strategies:
  - numerical scale
  - MoSCoW scale

- Necessary requirements (MUST)
  - They have to be necessarily implemented
- Desirable requirements (SHOULD)
  - It would be better implementing them but they are not necessary
- Possible requirements (COULD)
  - To be considered only in case of time and resources available
- Will not have (WON'T)
  - Not to be implemented

Requirements Engineering

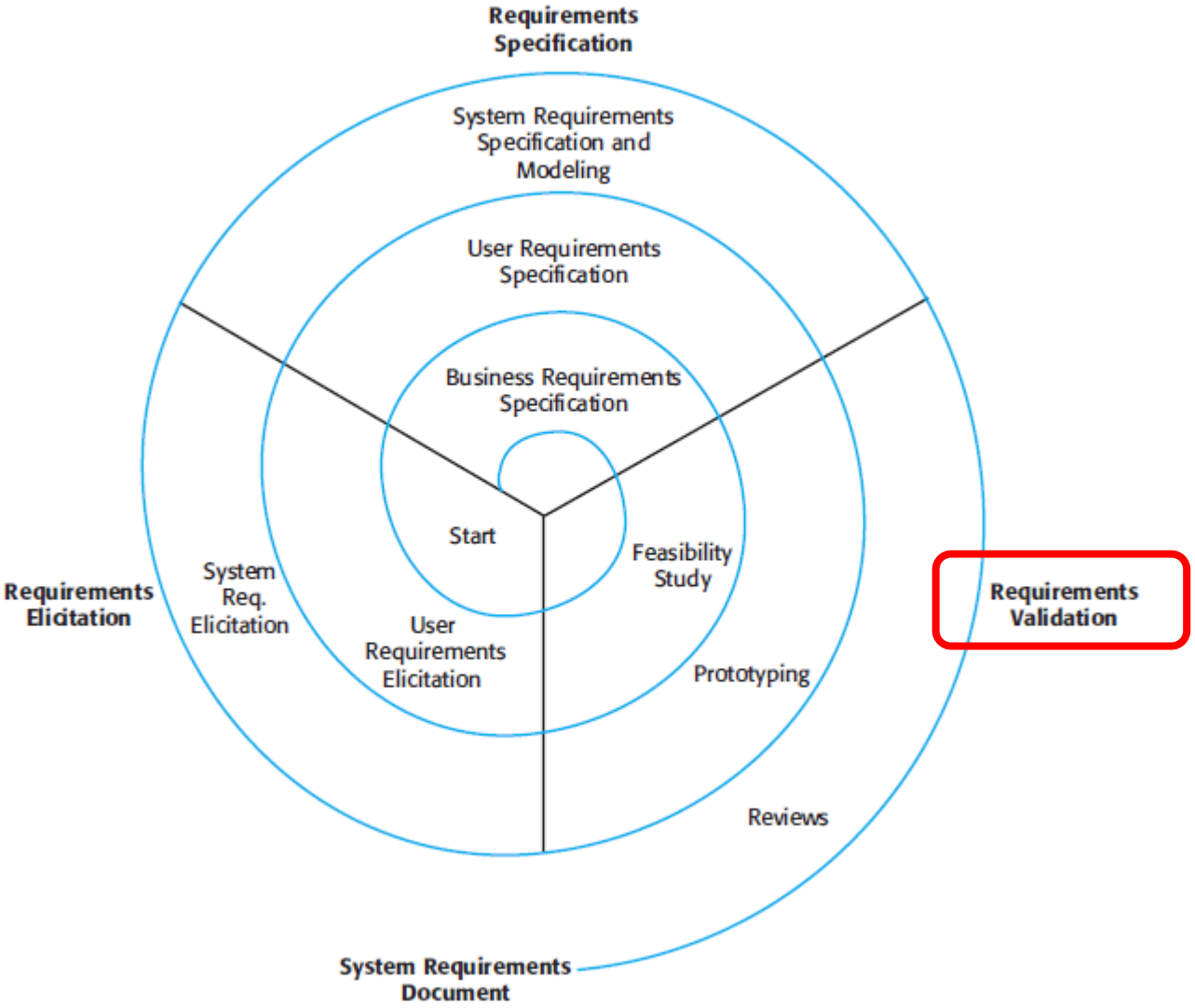


Requirement specification

Requirements specification

- The process of writing down the user and system requirements in a requirements document.
- User requirements have to be understandable by end-users and customers who do not have a technical background.
- System requirements are more detailed requirements and may include more technical information.
- The requirements may be part of a contract for the system development
  - It is therefore important that these are as complete as possible

Requirements Engineering



Requirement validation

Requirements validation

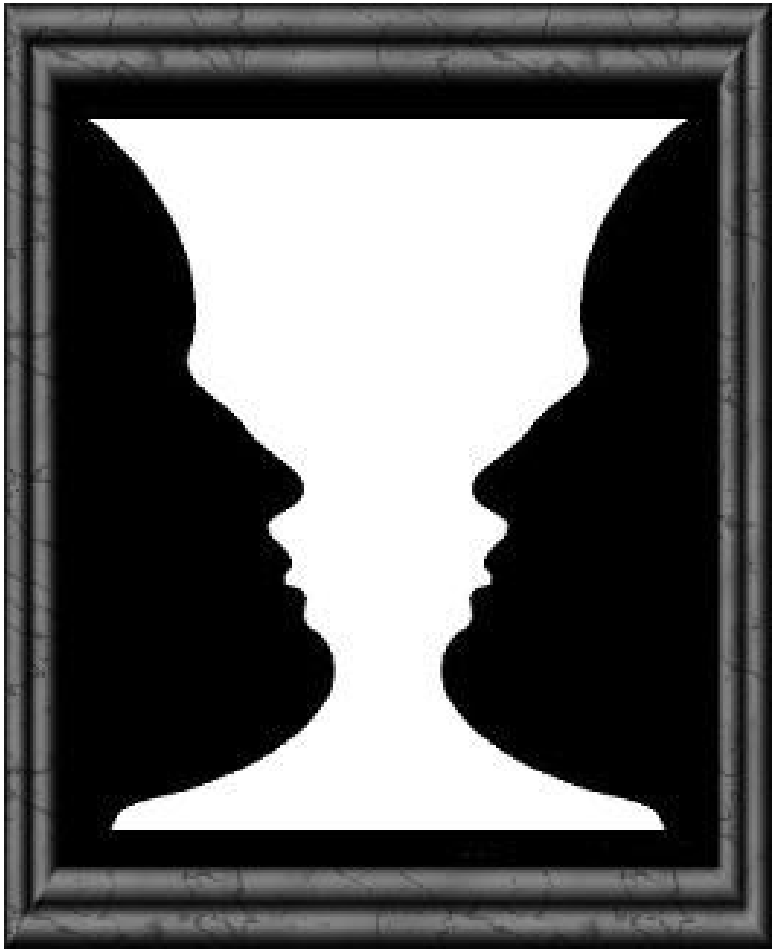
- Concerned with demonstrating that the requirements define the system that the customer really wants.
- Requirements error costs are high so validation is very important
  - Fixing a requirements error after delivery may cost up to 100 times the cost of fixing an implementation

Requirement properties

- Validity. Does the system provide the functions which support the customer's needs? Are there wrong or useless requirements?
- Consistency. Are there any requirements conflicts?
- Completeness. Are all functions required by the customer included?
- Realism. Can the requirements be implemented given available budget and technology

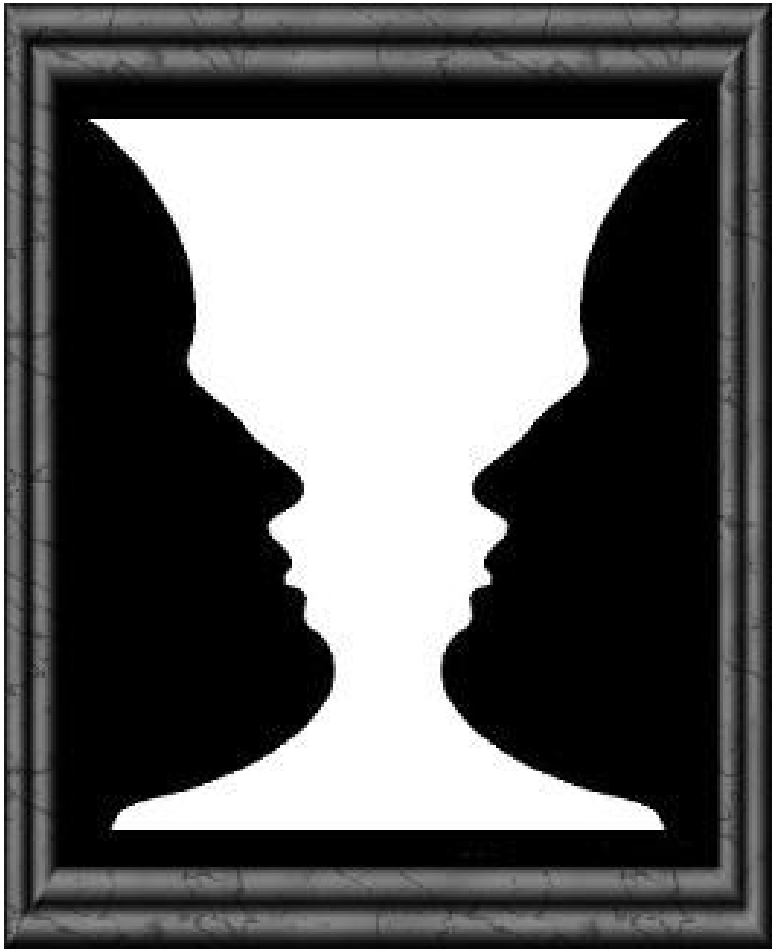
Is the requirement document conformant to the user needs?

Ambiguity



- What do you see?

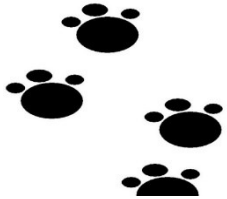
Ambiguity



- What do you see?
- Consider the term ‘search’: *“A user shall be able to search the appointments lists for all clinics”*
  - User intention – search for a patient name across all appointments in all clinics;
  - Developer interpretation – search for a patient name in an individual clinic. User chooses clinic then search.

Requirement properties

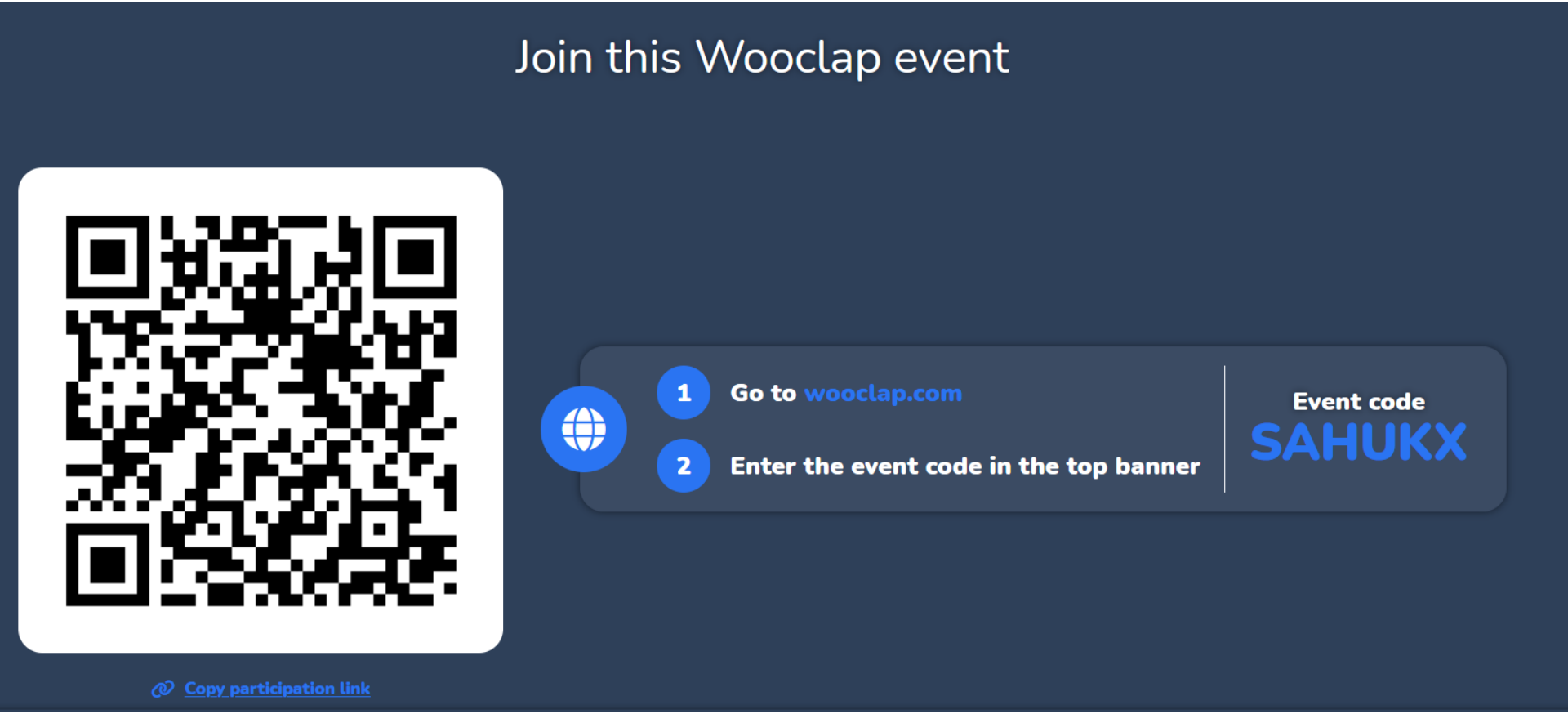
- Comprehensibility/Non-ambiguity
  - Is the requirement properly understood?
- Verifiability
  - Can the requirements be checked?
  - Is the requirement realistically testable?
- Traceability
  - Is the origin of the requirement clearly stated?
- Adaptability
  - Can the requirement be changed without a large impact on other requirements?



Requirements validation techniques

- Requirements reviews
  - Systematic manual analysis of the requirements.
- Prototyping
  - Using an executable model of the system to check requirements.
- Test-case generation
  - Developing tests for requirements to check testability.

Let’s try a test



Today

- Requirements engineering
- Problems with requirements
- Forms of requirements

What are the problems with requirements?

Ambiguity

- Problems arise when requirements are not precisely stated.
- Ambiguous requirements may be interpreted in different ways by developers and users.
- Consider the term ‘search’: A user shall be able to search the appointments lists for all clinics
  - User intention – search for a patient name across all appointments in all clinics;
  - Developer interpretation – search for a patient name in an individual clinic. User chooses clinic then search.

https://agile-od.com/mmdojo/2534/shared-understanding

https://agile-od.com/mmdojo/2534/shared-understanding

https://agile-od.com/mmdojo/2534/shared-understanding

https://agile-od.com/mmdojo/2534/shared-understanding

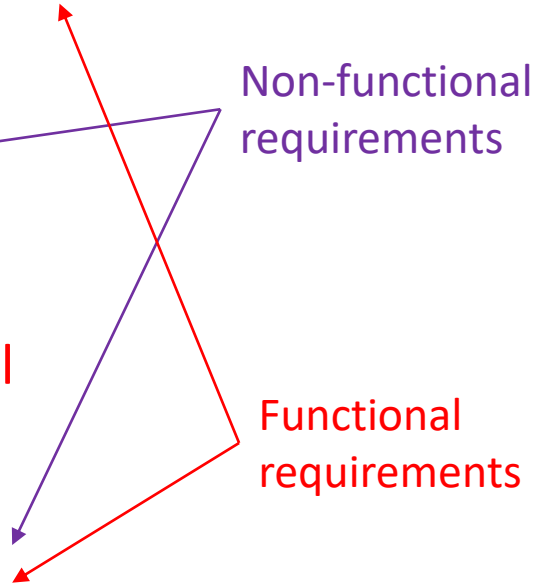
Other typical issues with the requirements document

- Lack of clarity
- Verbosity
- Use of technical terms
- Hidden assumptions
- Requirements that are too general or too detailed
- Parts of the solution (design) together with the requirements
- Two or more requirements are specified as a single requirement
- Different types of requirements blended together

An example

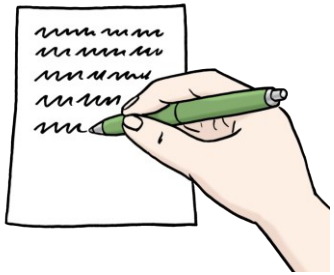
- Different types of requirements together
- More requirements in the same sentence

- Consider an ATM system. The service must provide the withdrawal, balance and account statement functions. The system must be available to people with disabilities, must guarantee a response time of less than a minute, and must be developed on the X86 architecture with an operating system compatible with that of the Bank. Withdrawal operations must require authentication via a secret code stored on the card. The system must be easily expandable, and adaptable to future banking needs. …

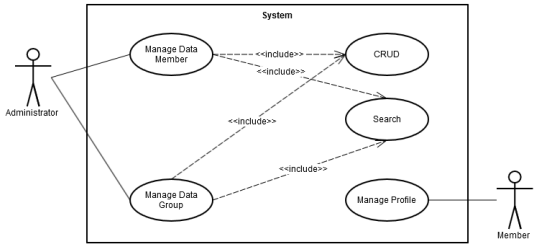


Today

- Requirements engineering
- Problems with requirements
- Forms of requirements



Forms of requirements

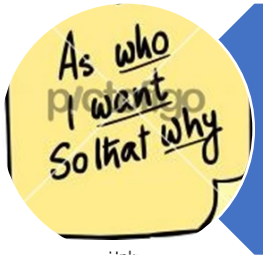


Different ways for writing down requirements

No requirements	• Code and fix
Plan-driven	• Formal languages • Visual notations • Structured or unstructured text • Use cases
Agile methods	• User stories

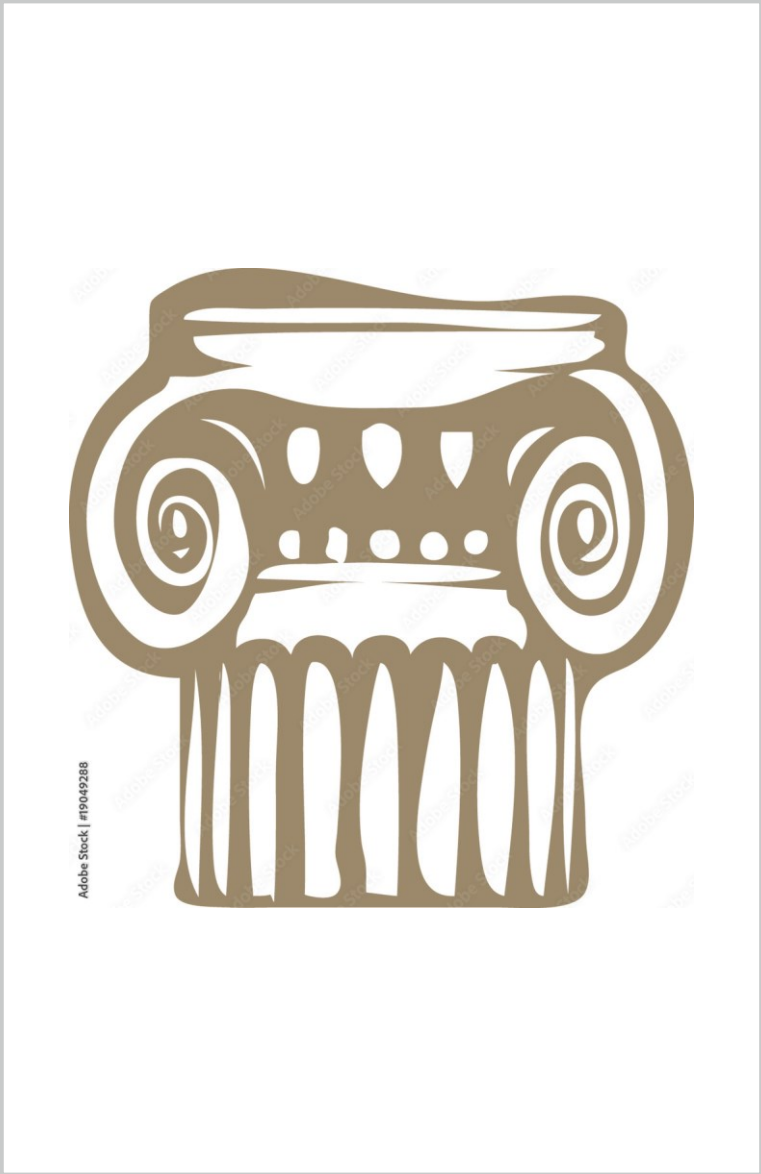
Requirement forms

Classical



User story

Use case



Classic requirement

Requirement template

<id> the system has to <function>

R1. The system has to manage all the cash registers of the shop (no more than 20)

R2. The system has to print the summary of the incomes of the day

R3. At the end of the day, the system has to list the items that have to be refurnished based on the items sold during the day

Examples

- Functional requirements related to an anonymous user

RF1: L'applicazione deve fornire agli utenti anonimi una breve spiegazione riguardo le funzionalità dell'applicazione Web e sul progetto.
RF2: L'applicazione deve poter fornire ad un utente anonimo la possibilità di consultare i le communities esistenti.
RF3: L'applicazione deve fornire all'utente anonimo una pagina dedicata alla registrazione o all'autenticazione

Examples

- Non-functional requirements related to performance and scalability

RNF5 Prestazioni:

- La velocità di risposta alla funzione di ricerca **RF6** deve essere inferiore ai 5 secondi
- La funzione di creazione di nuovi gruppi **RF13** deve essere eseguita dal sistema in al più 3 secondi

Le velocità di risposta o di azione vengono misurate a partire dal ricevimento della richiesta dal server.

La misurazione è effettuata nella rete locale del server.

RNF6 Scalabilità:

- Il sistema deve essere in grado di mantenere le prestazioni descritte in **RNF5** anche con 200 richieste contemporanee da parte degli utenti
- L'applicazione deve essere ben progettata per l'aggiunta di nuove funzionalità in futuro

User stories

Stories and scenarios

- User stories are real-life examples of how a system can be used
- A description of how a system may be used for a particular task
- Since they are based on a practical situation, stakeholders can relate to them and can comment on their situation with respect to the story.

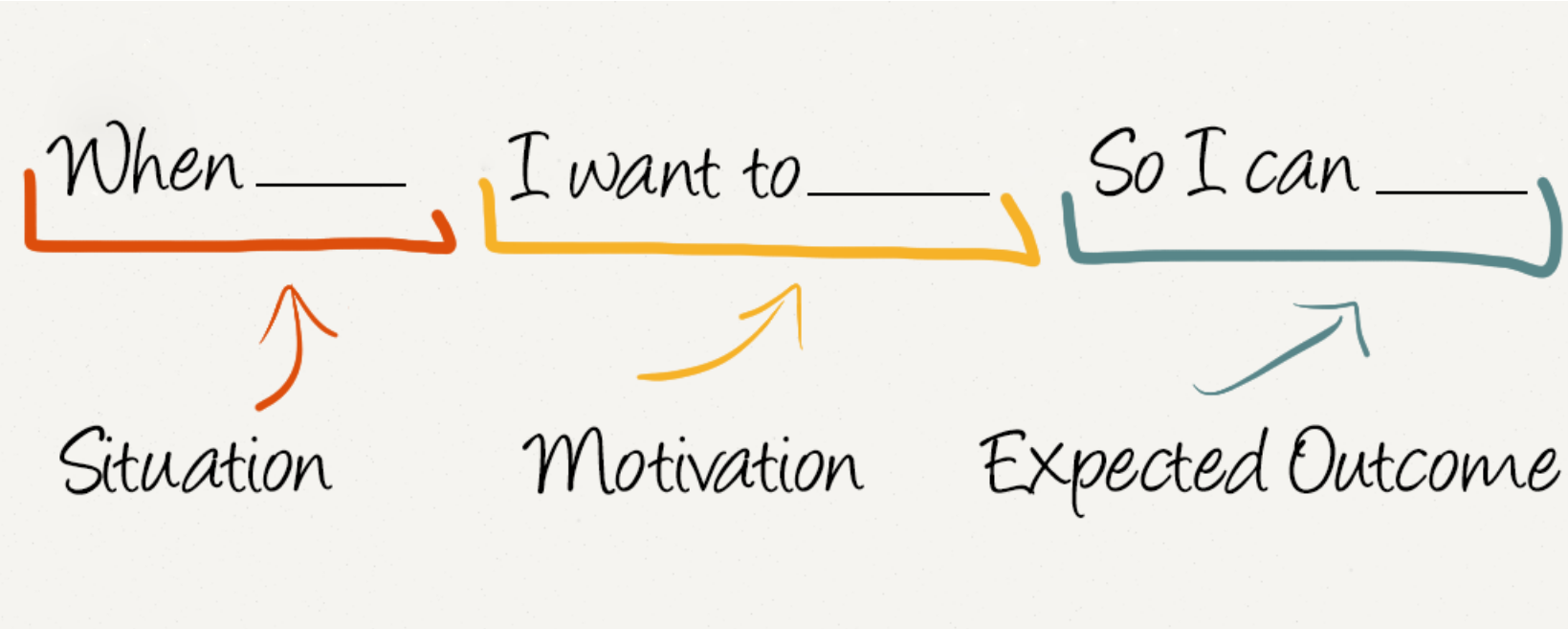
User stories

- In agile models, the requirements are written as user stories

  As a <user type> (role)
  I want <to do something> (objective)
  So that <I get some value> (value)

  As a registered user
  I want to look at the catalogue
  So that I can borrow a book

User stories



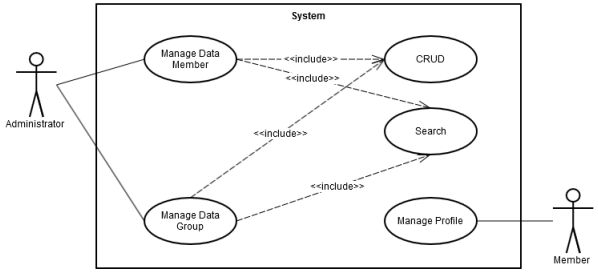
Why do we need user stories?

- The Product Owner writes the user stories and their “Definition of Done”, that is the acceptance criterion
- The user stories should be small enough to be realized in one or two weeks
- The development team chooses a user story to develop and set up the material so that the Product Owner can accept the resulting software, as reported in the Definition of Done

Examples

- User stories related to an anonymous user

- Come utente non registrato voglio poter accedere al sito per vedere di cosa si tratta
- Come utente non registrato voglio poter creare un account dove salvare le mie preferenze per ricevere suggerimenti adeguati, le mie interazioni, e i metodi per contattarmi



Use cases

Requirements with UML: use cases

- Proposed by Ivar Jacobson in 1992 for the Objectory method and used also in RUP

- Use cases are operating scenarios of the system users
- Scenarios are ways in which the system can be used (they define the operations or functions that the system offers to its users)

Use cases

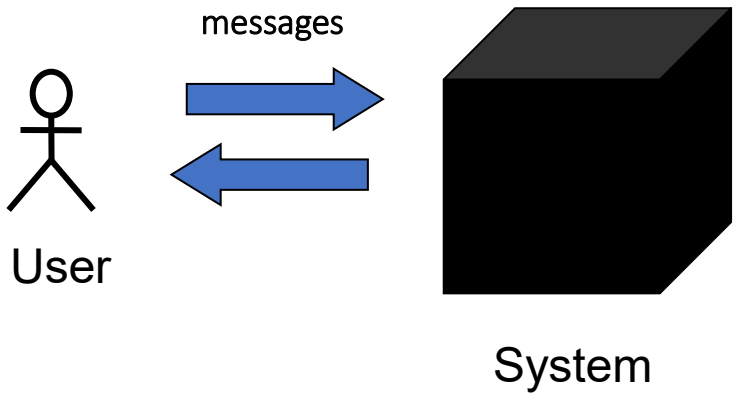
- Use-cases are a scenario-based technique in the UML which identify the actors in an interaction and which describe the interaction itself.
- A set of use cases should describe all possible interactions with the system.
- High-level graphical model supplemented by more detailed tabular description.
- Sequence diagrams may be used to add detail to use-cases by showing the sequence of event processing in the system.

Use case

- It represents a functional requirement
- It makes clear what you expect from a system ("what?")
- It hides the behaviour of the system ("how?")
- It is a sequence of actions (with variants) that produce a result observable by an actor

Use case

- Express the functional requirements of a system from the user's point of view
- The system is seen as a black-box

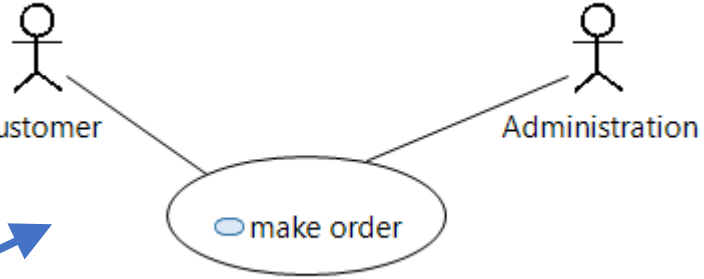


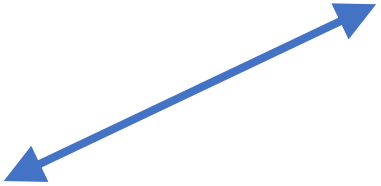
EXAMPLE ATM
The user selects the amount to withdraw and confirms the selected amount
The system checks that the amount is available and dispenses the banknotes

- Use cases express the interaction between the entities that interact with the system itself (called actors)
- Interaction = exchange of messages

Useful for …?

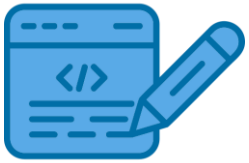
User requirements

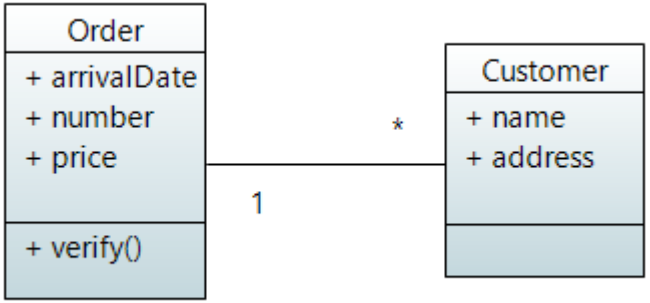




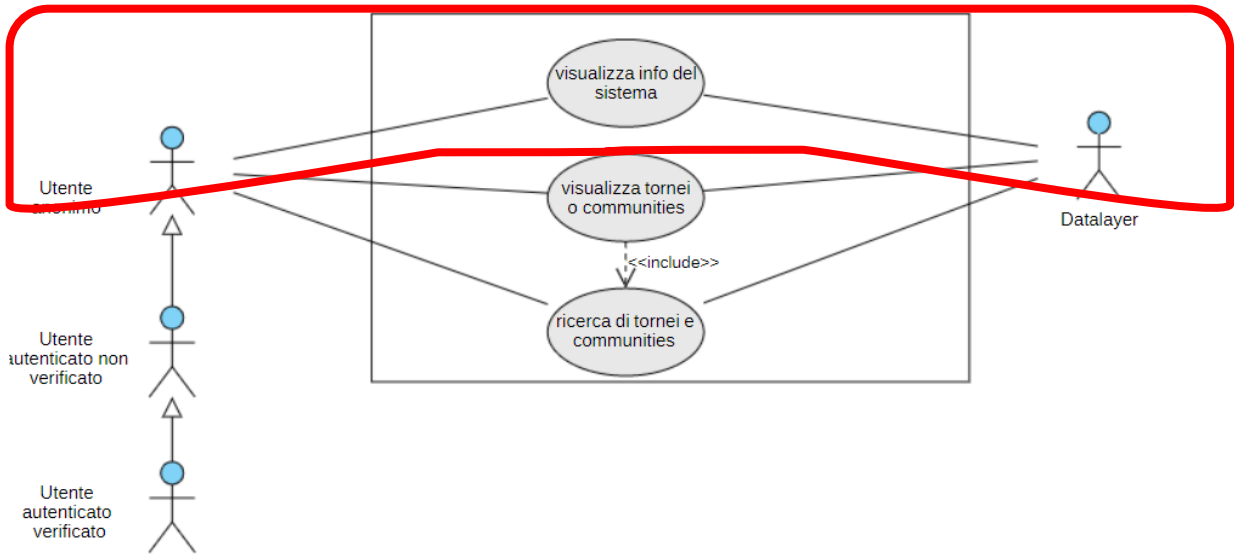
Software components

Test cases





Examples

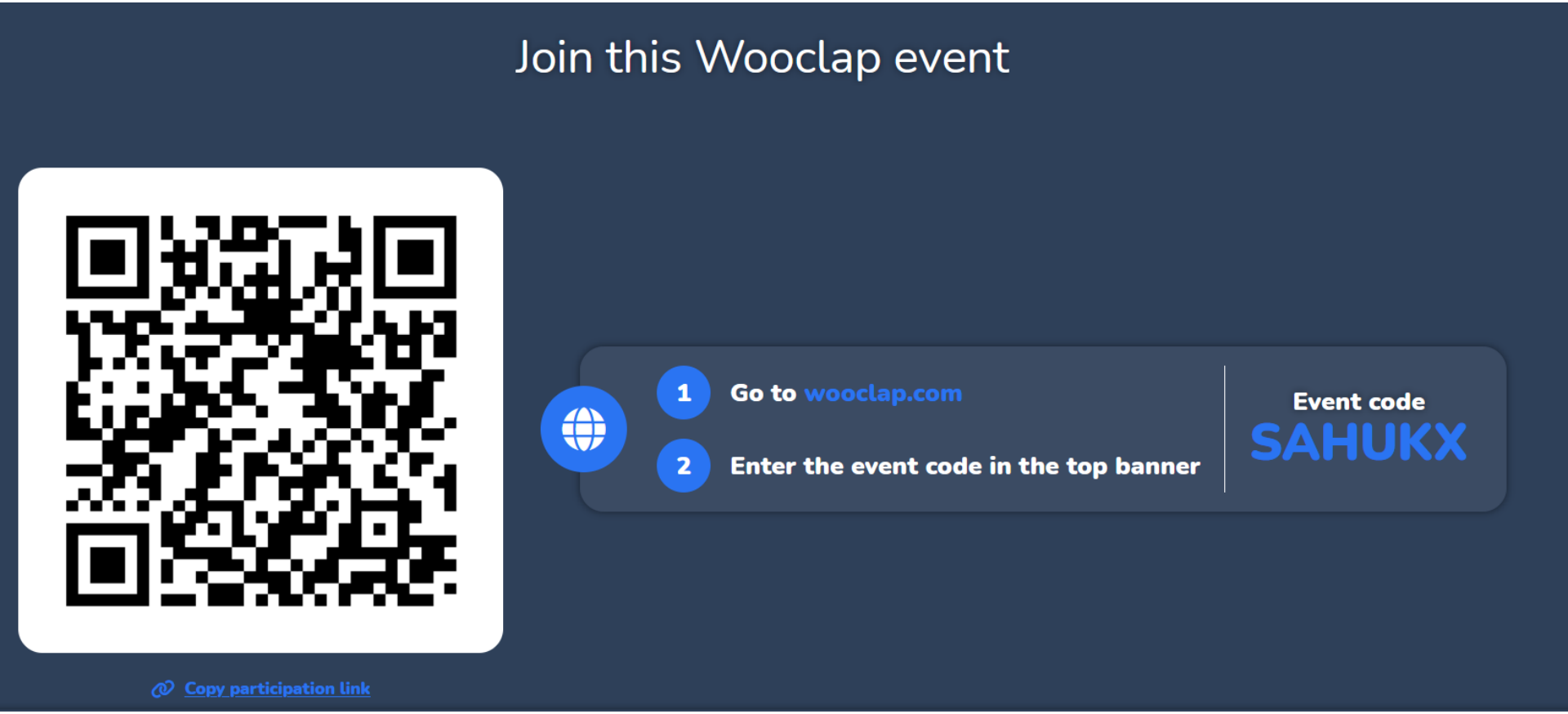


ID	UC1.1
nome	Visualizza info del sistema
attori	Utente non autenticato, Utente autenticato
descrizione	Questo use case descrive la visualizzazione di informazioni riguardo le funzionalità della web app nella schermata Home
evento scatenante	Accesso alla pagina Home del sito
precondizioni	L'utente sta visualizzando la schermata Home
flusso normale	L'utente visualizza una breve descrizione delle funzionalità del sito

Requirement forms

- Classical: the application will manage book loans for registered user
- User story: As a registered user, I want to look at the catalogue in order to borrow a book
- Use case: A registered user queries the function Catalogue for searching a volume, the catalogue asks for the search information

Let’s try a test



Summary

- Requirements engineering
- Problems with requirements
- Forms of requirements

Next time

- Modeling languages
- Use cases and use case diagrams